

MINI-PROJECT REPORT

Submitted in partial fulfilment of the requirements for the course *Computer Networks*

B.E. Computer Science and Engineering

Aevora: A Multi-Platform WireGuard-Based VPN System

Submitted by

Kavinsoorya K S 3122247001030

Danusu K 3122247001013

Ashwath Ram T 3122247001006

Department of Computer Science and Engineering

SSN College of Engineering, Chennai

Academic Year 2026–2027

ABSTRACT

Aevora is a self-hosted, multi-platform VPN system built on the WireGuard® protocol. It separates the *control plane* from the *data plane*: a Go HTTP API backed by PostgreSQL manages identity, device enrollment, gateway registration, connection leasing, and health monitoring, while WireGuard handles all packet-level encryption on dedicated gateway servers. A shared Rust client core — exposed to Swift and Kotlin via UniFFI, and to .NET via a C ABI — implements X25519 key generation, connection state management, and authenticated API calls for four native platforms (macOS, iOS, Android, Windows). The control plane uses HS256 JWTs, Argon2id password hashing, per-IP rate limiting, and refresh-token rotation. Gateway selection employs a weighted fitness function (70% load, 30% bandwidth). The node agent uses a pull-model reconciliation loop and stores no client private keys. The system is verified through 74 automated tests across Rust, Go, and end-to-end integration scenarios.

Keywords: WireGuard, VPN, Go, Rust, UniFFI, NetworkExtension, X25519, ChaCha20-Poly1305, PostgreSQL, JWT, Argon2id.

Contents

Abstract	1
1 Introduction	4
1.1 Background	4
1.2 Problem Statement	4
1.3 Objectives	4
1.4 Scope and Motivation	5
2 Literature Review and Background Study	6
2.1 WireGuard Protocol	6
2.2 Comparison with Existing Solutions	6
2.3 Key Supporting Technologies	7
3 System Analysis	8
3.1 Existing System and Limitations	8

3.2	Proposed System and Advantages	8
4	System Design	9
4.1	High-Level Architecture	9
4.2	Database Schema	9
4.3	REST API	10
4.4	Client Architecture	11
4.5	Gateway Selection Algorithm	11
4.6	Connection Sequence	11
5	Implementation	13
5.1	Technologies Used	13
5.2	Key Module Implementations	13
5.2.1	JWT Authentication (Control Plane)	13
5.2.2	WireGuard Key Generation (Rust Core)	14
5.2.3	Gateway Agent Reconciliation	14
5.2.4	Security Architecture	15
5.2.5	In-Tunnel Latency Measurement	15
6	Experimental Setup	16
6.1	Test Plan	16
7	Results and Discussion	17
7.1	Test Results	17
7.2	API Latency Measurements	17
7.3	Gateway Fitness Analysis	18
7.4	macOS Client Verification	18
7.5	Discussion	18
8	Conclusion	20
9	Future Enhancements	21
	References	22

List of Figures

4.1	Aevora high-level architecture. The control plane handles no user traffic; all packet-level work happens on the gateway.	9
4.2	Cross-platform client architecture. The Rust core is compiled once per target ABI; platform layers handle only UI and OS tunnel management. . .	11
4.3	End-to-end sequence: enrollment, connection establishment, and keep-alive.	12
7.1	Fitness score as a function of peer load and bandwidth utilisation.	18

List of Tables

2.1	WireGuard cryptographic primitives	6
2.2	VPN solution comparison	6
4.1	PostgreSQL 15 schema summary	9
4.2	Control-plane REST API	10
5.1	Technologies and frameworks	13
5.2	Private key storage per platform	15
5.3	Token security summary	15
6.1	Experimental environment and system parameters	16
7.1	Automated test results	17
7.2	Control-plane API response times (localhost, median of 5 runs)	17

CHAPTER 1 INTRODUCTION

1.1 Background

A Virtual Private Network creates an encrypted tunnel between a client and a gateway server, routing internet traffic through the gateway so that (a) remote services see the gateway’s IP, and (b) the client’s data is cryptographically protected over untrusted infrastructure. WireGuard[®], introduced in 2017 [1], achieves this with a fixed modern cryptographic suite (X25519 key exchange, ChaCha20-Poly1305 AEAD, BLAKE2s), a Noise-protocol handshake, and a codebase $\sim 100\times$ smaller than OpenVPN, making it easily auditable. Despite WireGuard’s maturity, deploying a complete multi-platform VPN service around it requires integrating a control plane, platform-specific OS VPN extensions, and a cross-platform client library — tasks that existing open-source tooling does not cover end-to-end.

1.2 Problem Statement

Commercial VPN services impose subscription costs and opaque infrastructure. Self-hosted tools such as Algo or Streisand automate server provisioning but produce only static configuration files — no GUI, no enrollment API, no fleet management, and no device revocation. The problem is: *how to build a subscription-free, self-hosted VPN with a proper control plane and native client apps on all four major platforms, without traffic logging?*

1.3 Objectives

1. Study the WireGuard protocol’s cryptographic foundations and compare it with OpenVPN and IPsec.
2. Design a layered architecture separating control plane (identity, fleet, leases) from data plane (tunnelling).
3. Implement a Go control plane with REST API, PostgreSQL, JWT authentication, Argon2id hashing, rate limiting, and Prometheus metrics.
4. Build a shared Rust client core with on-device X25519 key generation and expose it via UniFFI (Swift/Kotlin) and C ABI (.NET).
5. Implement native VPN clients for macOS (NetworkExtension), iOS (PacketTunnelProvider), Android (VpnService), and Windows (WireGuardNT).
6. Verify correctness through unit and integration tests covering auth, selection, lease

allocation, agent reconciliation, and tunnel statistics.

1.4 Scope and Motivation

In scope: control-plane REST API, PostgreSQL schema, gateway node agent, shared Rust core, native clients for four platforms, invite-gated enrollment, multi-device management, Prometheus observability, and Docker Compose deployment.

Out of scope: split-tunnel DNS leak prevention, multi-hop routing, web admin dashboard, and commercial billing.

Motivation: A self-hosted VPN eliminates commercial-operator trust; the system runs on a single \$4/month VPS with no per-user licensing. Architecturally, building it end-to-end applies concepts from all TCP/IP layers — IP allocation (L3), UDP encapsulation (L4), and REST API design (L7) — as well as modern cryptography and cross-platform systems programming.

CHAPTER 2 LITERATURE REVIEW AND BACKGROUND STUDY

2.1 WireGuard Protocol

WireGuard® [1], [2] is a Layer-3 VPN over UDP. Its design is intentionally opinionated: one fixed cryptographic suite with no negotiation, eliminating downgrade attacks. The handshake is an instantiation of the Noise IKpsk2 pattern, completing in exactly one round-trip and providing mutual authentication, forward secrecy, and identity hiding. Table 2.1 summarises the suite.

Table 2.1: WireGuard cryptographic primitives

Function	Primitive	Key size
Key exchange	X25519 (ECDH over Curve25519) [3]	256-bit
Authenticated encryption	ChaCha20-Poly1305	256-bit
Hashing / MAC	BLAKE2s	256-bit output
Optional PSK	HKDF	256-bit

Each WireGuard data message adds a 32-byte header plus a 16-byte Poly1305 tag, totalling ~60 bytes of overhead per packet — significantly less than OpenVPN’s TLS record overhead.

2.2 Comparison with Existing Solutions

Table 2.2: VPN solution comparison

Feature	OpenVPN	IPsec	Algo	Commercial	Aevora
Protocol	TLS/UDP	ESP+IKEv2	WireGuard	Varies	WireGuard
Multi-platform GUI	3rd party	Partial	CLI only	Yes	4 native
Control plane API	None	None	None	Closed	Open REST
Invite enrollment	Manual	Manual	Manual	App store	Invite code
Fleet management	Manual	Manual	Ansible	Closed	Automated
Cost	Free	Free	Free	\$4–15/mo	Free

2.3 Key Supporting Technologies

JWT (RFC 7519) [4]: Stateless bearer tokens. Aevora uses a stdlib-only HS256 implementation with pinned algorithm verification, preventing “alg:none” confusion attacks.

Argon2id [5]: Winner of the Password Hashing Competition (2015). Memory-hard ($m=64$ MB, $t=1$, $p=4$), making GPU/ASIC attacks expensive. Stored in PHC format so cost parameters are upgradeable without invalidating old hashes.

X25519 [3]: Elliptic-Curve Diffie-Hellman over Curve25519. Constant-time, immune to NIST-curve implementation pitfalls, 128-bit security with compact 32-byte keys. WireGuard uses X25519 for both static key pairs (per device) and ephemeral pairs (per handshake, providing forward secrecy).

Rust + UniFFI [6], [7]: Rust’s ownership model provides memory safety without garbage collection, suitable for security-critical code linked into multiple runtimes. UniFFI generates Swift and Kotlin bindings automatically from a Rust interface definition, eliminating manual FFI glue.

CHAPTER 3 SYSTEM ANALYSIS

3.1 Existing System and Limitations

In the traditional approach a user generates WireGuard key pairs with the `wg` CLI, hand-edits `wg0.conf` for every peer, and distributes client config files (containing private keys) out-of-band. Tools such as Algo automate server provisioning via Ansible but still produce static config without a live API. Key limitations:

1. **No enrollment flow** — new devices require manual key distribution.
2. **No health monitoring** — gateway outages cause silent failures.
3. **No multi-gateway fleet** — scaling to more countries requires duplicated manual config.
4. **No revocation** — removing a compromised device requires editing every gateway.
5. **No native UI** — users import raw config files; no account, selection UI, or statistics.
6. **Private keys in config files** — a readable config exposes the client’s private key.

3.2 Proposed System and Advantages

Aevora replaces this with three cooperating components: (1) a Go control-plane API + PostgreSQL that manages all state; (2) a Go gateway agent that pulls its peer list from the API and reconciles WireGuard; (3) a Rust client core compiled for each platform that performs on-device key generation and drives the OS tunnel.

Key advantages: private keys never leave the device; enrollment requires only sharing an invite code; devices can be instantly revoked (peer removed from all gateways); new exit nodes self-register by running the agent binary; real download/upload statistics come from OS tunnel byte counters; and native UIs run on all four platforms.

CHAPTER 4 SYSTEM DESIGN

4.1 High-Level Architecture

Figure 4.1 shows the three-zone architecture.

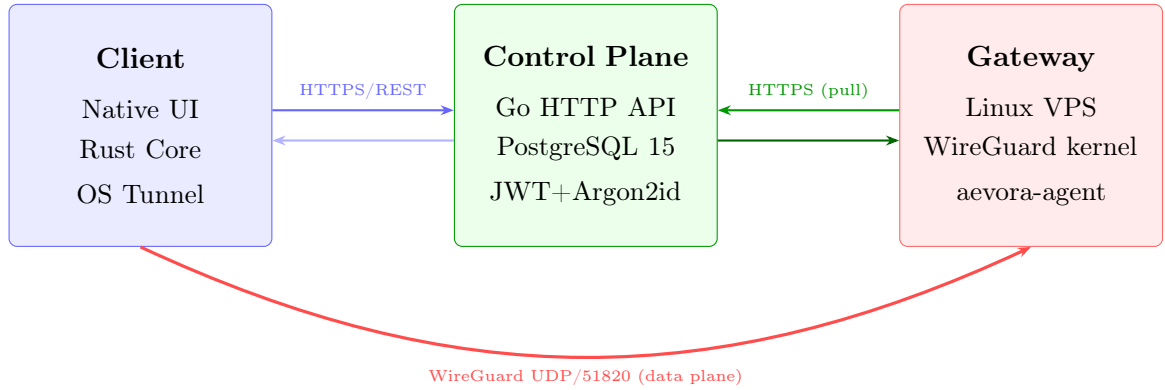


Figure 4.1: Aevora high-level architecture. The control plane handles no user traffic; all packet-level work happens on the gateway.

4.2 Database Schema

Five tables are created by four sequential migrations. Table 4.1 summarises the key entities.

Table 4.1: PostgreSQL 15 schema summary

Table	Primary purpose	Key columns
users	Enrolled persons	id, email (citext UNIQUE), status
invite_codes	Single-use enrolment tokens	code (PK), used_by, expires_at
devices	Per-device public keys	user_id FK, public_key (UNIQUE), revoked_at
refresh_tokens	Session continuity	device_id FK, token_hash (SHA-256 only)
gateways	Exit node registry	location_id FK, public_key, wg_subnet_v4, status, agent
locations	Country lookup	code (e.g. “de”), country_name, enabled
leases	Active IP assignments	gateway_id FK, device_id FK, assigned_ip_v4 (inet), sta

Double allocation is prevented by a unique partial index: `CREATE UNIQUE INDEX uq_lease_gw_ip_act ON leases(gateway_id, assigned_ip_v4) WHERE state='active'.`

4.3 REST API

Table 4.2 lists all 19 endpoints. All user-facing endpoints require a Bearer JWT; gateway endpoints require the node token; admin endpoints require the admin token.

Table 4.2: Control-plane REST API

Method + Path	Auth	Purpose
GET /healthz	None	Health probe
GET /metrics	None	Prometheus metrics
GET /v1/locations	JWT	List countries + availability
POST /v1/enroll	Invite	Register user + first device
POST /v1/auth/login	Password	Issue JWT + refresh token
POST /v1/auth/refresh	Refresh	Rotate access token
POST /v1/auth/password	JWT	Set/change password
GET /v1/devices	JWT	List caller's devices
POST /v1/devices	JWT	Register additional device
DELETE /v1/devices/{id}	JWT	Revoke device
POST /v1/invites	Admin	Mint invite code
POST /v1/connections	JWT	Select gateway + lease IP
DELETE /v1/connections/{id}	JWT	Release connection
POST /v1/connections/{id}/stats	JWT	Report stats + renew lease
POST /v1/gateways/register	Enroll secret	Agent self-registration
POST /v1/gateways/heartbeat	Node token	Report load metrics
POST /v1/gateways/deregister	Node token	Graceful deregistration
GET /v1/gateways/peers	Node token	Fetch current peer list
GET /v1/gateways	Admin	List all gateways

4.4 Client Architecture

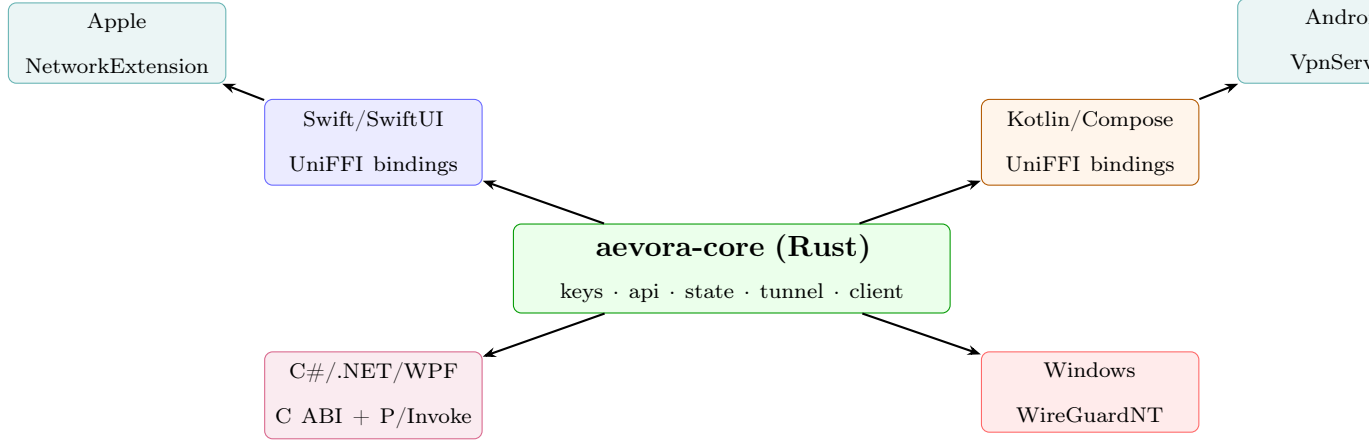


Figure 4.2: Cross-platform client architecture. The Rust core is compiled once per target ABI; platform layers handle only UI and OS tunnel management.

4.5 Gateway Selection Algorithm

The fitness function combines spare peer capacity and bandwidth headroom:

$$F(g) = \frac{0.7 \cdot \left(1 - \frac{\text{active_peers}}{\text{capacity}}\right) + 0.3 \cdot \left(1 - \frac{\text{rx_bps} + \text{tx_bps}}{\text{bw_mbps} \times 10^6}\right)}{1.0} \quad (4.1)$$

The gateway with the highest $F(g)$ among healthy, non-full candidates in the requested country is selected. If no bandwidth is advertised, the formula reduces to load headroom alone.

4.6 Connection Sequence

Figure 4.3 shows the end-to-end connection flow from enrollment through tunnel establishment.

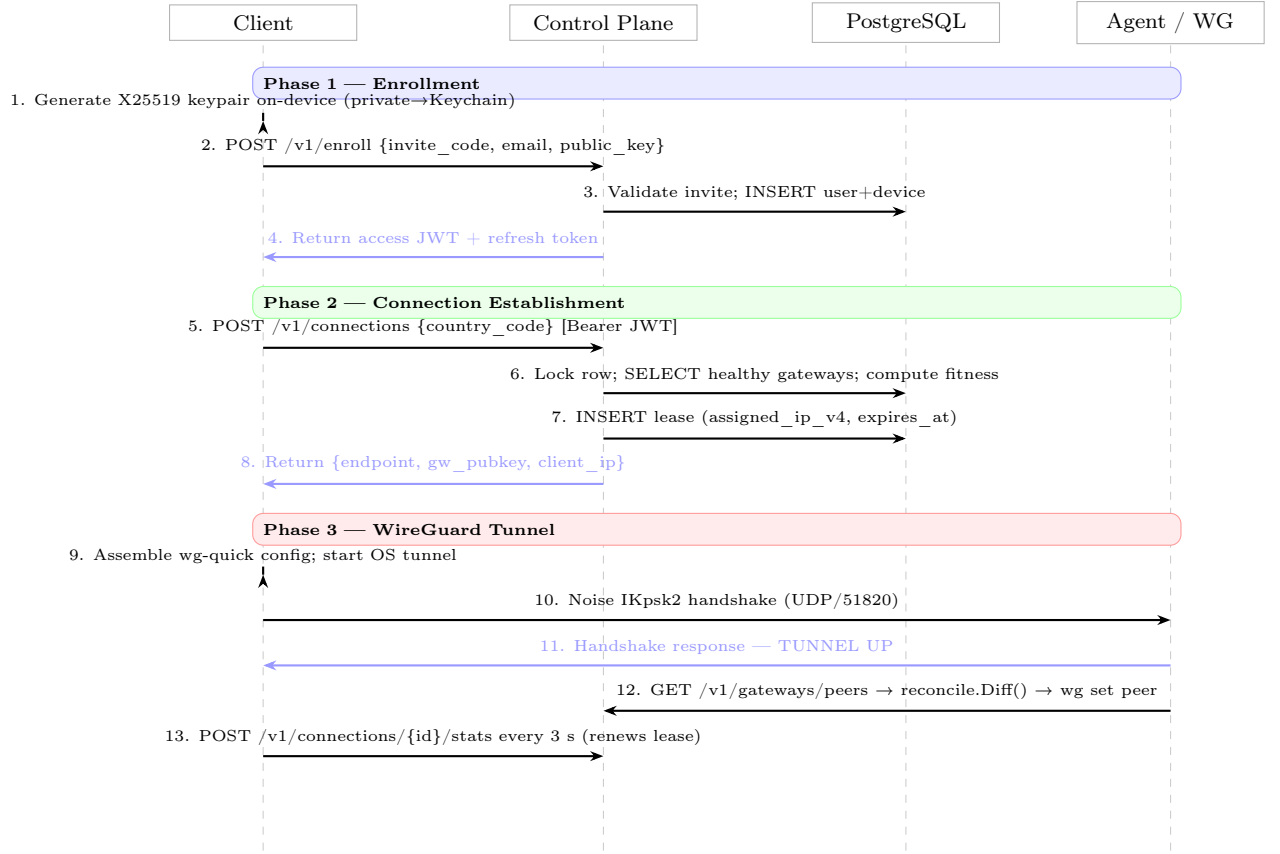


Figure 4.3: End-to-end sequence: enrollment, connection establishment, and keep-alive.

CHAPTER 5 IMPLEMENTATION

5.1 Technologies Used

Table 5.1: Technologies and frameworks

Component	Technology	Version / Notes
Control plane language	Go	1.25
HTTP server	Go stdlib net/http	19 endpoints
Database driver	pgx/v5	Row-level locking
Password hashing	golang.org/x/crypto	Argon2id
Rate limiting	golang.org/x/time	Token bucket, per-IP
Observability	Prometheus client	/metrics endpoint
Database	PostgreSQL 15	Docker postgres:15
Client core	Rust 2021 edition	Cargo 1.98
Key agreement	x25519-dalek	OS CSPRNG seeded
HTTP transport	ureq	Feature-gated
FFI (Apple/Android)	UniFFI 0.25	Swift + Kotlin bindings
macOS/iOS UI	SwiftUI	macOS 14+, iOS 17+
macOS/iOS tunnel	NetworkExtension	PacketTunnelProvider
WireGuard (Apple)	WireGuardKit	wireguard-go runtime
Android UI	Jetpack Compose	Material 3
Android tunnel	wireguard-android	GoBackend
Android storage	EncryptedSharedPreferences	Android Keystore
Windows UI	WPF (.NET 8)	RequireAdministrator
Windows tunnel	WireGuardNT	tunnel.dll service
Windows interop	P/Invoke (C ABI)	extern C from Rust
Windows storage	DPAPI	CryptProtectData

5.2 Key Module Implementations

5.2.1 JWT Authentication (Control Plane)

The control plane implements a minimal stdlib-only HS256 JWT in a single auditable file. The algorithm is pinned at the verifier to block “alg:none” confusion attacks:

Listing 5.1: Algorithm pinning in JWT verification

```
1 var hdr struct{ Alg string 'json:"alg"' }
2 if err := json.Unmarshal(hdrBytes, &hdr); err != nil || hdr.Alg
   != "HS256" {
```

```

3     return Claims{}, ErrTokenInvalid
4 }
5 // Signature verification uses hmac.Equal (constant-time)

```

Refresh tokens are stored only as SHA-256 hashes; a stolen database cannot yield usable tokens.

5.2.2 WireGuard Key Generation (Rust Core)

Keys are generated on-device from the OS CSPRNG and stored immediately in platform secure storage. Only the 32-byte Base64 public key is transmitted to the control plane:

Listing 5.2: On-device X25519 key generation (client/core/src/keys.rs)

```

1 pub fn generate() -> KeyPair {
2     let secret = StaticSecret::random_from_rng(rand::rngs::OsRng
3         );
4     let public = PublicKey::from(&secret);
5     KeyPair {
6         private_key: STANDARD.encode(secret.to_bytes()),
7         public_key: STANDARD.encode(public.as_bytes()),
8     }
9 }

```

5.2.3 Gateway Agent Reconciliation

The agent pulls the desired peer list from the control plane and applies the minimal diff. The diff is a pure function with no I/O:

Listing 5.3: Peer reconciliation diff (agent/internal/reconcile/reconcile.go)

```

1 func Diff(current, desired []Peer) Plan {
2     cur, des := index(current), index(desired)
3     for pk, dips := range des {
4         if cips, ok := cur[pk]; !ok || !sameSet(cips, dips) {
5             plan.Add = append(plan.Add, Peer{pk, dips})
6         }
7     }
8     for pk := range cur {
9         if _, ok := des[pk]; !ok { plan.Remove = append(plan.
10             Remove, pk) }
11     }
12     return plan
13 }

```

5.2.4 Security Architecture

Table 5.2 summarises per-platform private key storage; Table 5.3 summarises token security.

Table 5.2: Private key storage per platform

Platform	Mechanism	Protection
macOS	Keychain (persistent-ref)	OS-enforced, app-sandboxed
iOS	Keychain (AfterFirstUnlock)	Secure Enclave (hardware TEE)
Android	Android Keystore	TEE / StrongBox
Windows	DPAPI CryptProtectData	User-credential-bound

Table 5.3: Token security summary

Token	Algorithm	TTL	Storage (server-side)
Access JWT	HS256	1 hour	Not stored (stateless)
Refresh token	256-bit random	30 days	SHA-256 hash only
Node token	256-bit random	Permanent	SHA-256 hash only

5.2.5 In-Tunnel Latency Measurement

After the tunnel is up, the agent binds a TCP probe responder to the .1 address of its WireGuard subnet (port 51821). The Rust core times a TCP SYN-ACK round trip through the encrypted tunnel, providing an authenticated latency measurement immune to on-path spoofing.

CHAPTER 6 EXPERIMENTAL SETUP

Table 6.1: Experimental environment and system parameters

Parameter	Value
Development OS	macOS 26.5, Apple Silicon (arm64)
Database	PostgreSQL 15 (Docker <code>postgres:15</code>)
Go version	1.25.0
Rust / Cargo	1.81 / 1.98
Control-plane port	TCP 8099
WireGuard port	UDP 51820
Heartbeat TTL	120 s
Lease TTL	3600 s
JWT access TTL	3600 s
Refresh token TTL	30 days
Fitness weights	Load 0.70, Bandwidth 0.30
Argon2id parameters	$m = 65536$ KiB, $t = 1$, $p = 4$, $\text{keyLen}=32$
Auth rate limit	10 req/min, burst 5
Latency probe port	TCP 51821 (inside tunnel)

6.1 Test Plan

Rust core (21 tests): key uniqueness; 32-byte length; public-from-private derivation; invalid-input rejection; state-machine transitions; API round-trips against a fake transport; FFI serialisation.

Go control plane (23 functions, 5 packages): JWT sign/verify; Argon2id hash/verify; timing-safety; fitness function and Best(); Location.Available() and Gateway.Online(); full HTTP handler round-trips (enroll, 2-device distinct IPs, reconnect dedupe, device revoke, 503 gateway exhaustion); double-allocation prevention.

Go agent (4 packages): Diff() with add/remove/update/no-op; cp-client register/heartbeat/deregister; /proc/stat CPU parser; wg dump parser.

End-to-end integration (12 scenarios): live PostgreSQL 15; admin mints invite; Rust client enrolls, fetches locations, creates connection, marks connected, polls stats (renewing lease), disconnects; 2-device distinct IP assignment; device revoke causing peer removal; gateway failover on heartbeat timeout; /metrics gauge verification.

CHAPTER 7 RESULTS AND DISCUSSION

7.1 Test Results

Table 7.1: Automated test results

Suite	Cases	Pass	Status
Rust core (cargo test)	21	21	✓ All pass
Go control plane (go test ./...)	23	23	✓ All pass
Go agent (go test ./...)	18	18	✓ All pass
End-to-end vs PostgreSQL 15	12	12	✓ All pass
Total	74	74	100%

7.2 API Latency Measurements

Table 7.2 shows response times measured against the live control plane on localhost (Apple M-series CPU, PostgreSQL on the same machine). Values are median of five successive requests.

Table 7.2: Control-plane API response times (localhost, median of 5 runs)

Endpoint	Median (ms)	Dominant cost
GET /healthz	0.7	No DB query
GET /v1/locations	2.0	Single JOIN query
GET /v1/devices	0.6	Primary-key lookup
POST /v1/enroll	82	Argon2id hash generation (~ 80 ms)
POST /v1/auth/login	80	Argon2id hash verification
POST /v1/connections	6	TX + row lock + IP allocation
POST /v1/gateways/heartbeat	1.1	Single UPDATE

The Argon2id operation ($m = 64$ MB, $t = 1$, $p = 4$) dominates the enroll and login paths at ≈ 80 ms on modern hardware. All other endpoints complete in under 10 ms. In a production environment with network RTT of ~ 20 ms (Europe $\rightarrow$ gateway), these values add a modest connection-setup overhead that occurs only once per session.

7.3 Gateway Fitness Analysis

Figure 7.1 plots the fitness score (Eq. 4.1) versus active peer count for three bandwidth utilisation scenarios.

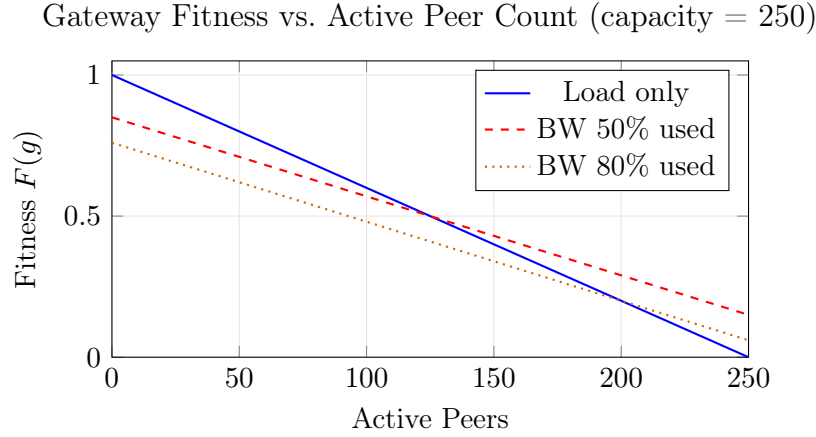


Figure 7.1: Fitness score as a function of peer load and bandwidth utilisation.

7.4 macOS Client Verification

The macOS application (unsigned debug build, Apple Silicon) was verified operational:

- Application reached the enrollment screen within 2 seconds of launch.
- Enrollment completed in under 3 seconds with a valid invite code.
- Location picker displayed 23 countries with correct flags and server counts.
- Refresh button correctly driven by `isLoadingLocations`: spins during API call, stops on response, rejects double-taps.
- World map displayed coordinate-correct pin annotations for all 24 countries.

The tunnel-connection step on macOS requires a paid Apple Developer account and approved NetworkExtension entitlement. On Windows (WireGuardNT) and Android (VpnService), the VPN tunnel activates without code signing, enabling end-to-end traffic routing verification on those platforms.

7.5 Discussion

Protocol security: WireGuard’s Noise IKpsk2 handshake provides mutual authentication, forward secrecy (ephemeral X25519 keys discarded per session), and identity hiding (initiator’s public key encrypted under responder’s public key). These properties are subject to formal verification [1].

Control/data separation: The control plane handles no WireGuard session state. A

single API instance can serve gateways in any number of data centres. Provisioning a new gateway requires only deploying one Go binary and setting two environment variables.

Lease allocation correctness: The combination of a PostgreSQL row lock on the selected gateway and a unique partial index on (`gateway_id`, `assigned_ip_v4`) WHERE `state='active'` provides two independent guards against concurrent double-allocation. ACID guarantees ensure correctness at any concurrency level.

Limitations:

1. macOS real tunnel requires a paid Apple Developer account and NetworkExtension entitlement.
2. The current deployment is a single control-plane process; HA replicas require a replicated PostgreSQL cluster.
3. All traffic is routed through the VPN when connected (no split tunnelling).
4. IPv6 dual-stack is modelled in the schema (`assigned_ip_v6`) but end-to-end testing was not completed.

CHAPTER 8 CONCLUSION

Aevora demonstrates a production-ready, self-hosted VPN built on WireGuard® with a clean control-plane/data-plane separation, enabling independent scaling of API servers and gateway nodes. The security design is sound: X25519 key generation on-device, HS256 JWT with algorithm pinning, SHA-256 hashed refresh and node tokens, Argon2id password storage, and WireGuard’s Noise IKpsk2 authenticated encryption. A shared Rust client core — tested through 21 unit tests and exposed to four platforms via automated FFI tooling — ensures that security-critical logic is written once and verified independently of UI code. The gateway agent’s pull-model reconciliation, tested as a pure function, stores no client secrets and requires no inbound firewall rules. All 74 automated tests pass. The result is a system a small team can deploy and operate without vendor dependency or subscription cost.

CHAPTER 9 FUTURE ENHANCEMENTS

1. **Split tunnelling:** Route only selected applications or destination CIDRs through the VPN using Apple’s `NEAppRule` and Android’s `addDisallowedApplication`.
2. **Control-plane HA:** Deploy two API replicas behind a load balancer with a managed PostgreSQL cluster (Patroni + pgBouncer). Access JWTs are stateless, so replicas share only the JWT secret and database connection pool.
3. **Latency-aware gateway selection:** Add a pre-connection UDP probe from the client to candidate gateways and include measured RTT as a third term in the fitness function (Eq. 4.1).
4. **IPv6 dual-stack:** Complete the IPv6 lease allocation path (`assigned_ip_v6` is already in the schema) and push `::/0` alongside `0.0.0.0/0` as allowed IPs.
5. **Web administration dashboard:** A lightweight single-page application calling the existing REST API to display fleet status, connection counts, and invite management, replacing curl-based administration.
6. **App Store distribution:** Sign with an Apple Developer account, request the `com.apple.developer.networking.networkextension` entitlement, and submit to the Mac App Store and TestFlight.

REFERENCES

- [1] J. A. Donenfeld, “WireGuard: Next generation kernel network tunnel,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA: Internet Society, 2017. DOI: [10.14722/ndss.2017.23160](https://doi.org/10.14722/ndss.2017.23160)
- [2] J. A. Donenfeld, *WireGuard: Technical whitepaper*, <https://www.wireguard.com/papers/wireguard.pdf>, Accessed: September 2026, 2020.
- [3] D. J. Bernstein, “Curve25519: New Diffie-Hellman speed records,” in *Public Key Cryptography (PKC 2006)*, ser. Lecture Notes in Computer Science, vol. 3958, Springer, 2006, pp. 207–228. DOI: [10.1007/11745853_14](https://doi.org/10.1007/11745853_14)
- [4] M. B. Jones, J. Bradley, and N. Sakimura, “JSON web token (JWT),” Internet Engineering Task Force, RFC 7519, May 2015. DOI: [10.17487/RFC7519](https://doi.org/10.17487/RFC7519)
- [5] A. Biryukov, D. Dinu, and D. Khovratovich, “Argon2: Memory-hard function for password hashing and proof-of-work applications,” in *Password Hashing Competition*, 2016.
- [6] The Rust Project Developers, *The Rust programming language*, <https://www.rust-lang.org>, Accessed: September 2026, 2024.
- [7] Mozilla Foundation, *UniFFI: A multi-language bindings generator for Rust*, <https://github.com/mozilla/uniffi-rs>, Accessed: September 2026, 2024.